AMERICAN UNIVERSITY OF BEIRUT

Module 3 - Lecture Notes

Frontend, Testing & Debugging with AI

AI-Assisted Coding

Module snapshot

Module 3 turns the Task Tracker backend from Module 2 into a working browser product. You build a Kanban board with ToDo, InProgress, and Done columns, cards sorted by priority, drag-and-drop status updates that persist through the API, and a create/edit modal with both client-side and server-side validation. You also practice the AI-assisted workflow in VS Code: ask for small changes, inspect suggestions or diffs, run the browser or tests, and refine from evidence.

By the end, you will have a working frontend, a behavior contract, a safe refactor, expanded backend tests, and a short debugging log.

How to use these notes

Read these notes alongside the Module 3 videos. They are not a transcript. They are a student-facing guide to the concepts, examples, verification habits, and deliverables taught in the module.

Learning outcomes

1. Explain how Module 3 shifts the Task Tracker from a backend API to a browser-based product surface.
2. Use VS Code with an AI coding assistant for inline suggestions, chat, and selected edits without blindly accepting output.
3. Build a Kanban board incrementally: static layout, styling, fetch/render logic, UI states, and drag-and-drop.
4. Connect a frontend to the FastAPI backend using GET /tasks and PATCH /tasks/{id}.
5. Diagnose and fix local browser-to-backend issues such as CORS when DevTools shows blocked requests.
6. Implement a create/edit modal form with title trimming, POST/PATCH submission, server 422 handling, and dismissal flows.

7. Refactor safely by writing a behavior contract, committing first, selecting one section at a time, and re-running the checklist.
8. Use AI to brainstorm backend edge cases, generate focused pytest tests, and prove tests are real through deliberate breakage.
9. Debug from evidence such as status codes, response bodies, console output, network requests, and pytest failures.
10. Reject AI suggestions that weaken tests, hide errors, or suppress symptoms instead of fixing the source cause.

Module outputs

Output	What it should contain	Why it matters
Verified VS Code + Copilot setup	VS Code opened on the Task Tracker repo, Copilot enabled, and at least one route summary checked against app/main.py.	Confirms the assistant is reading the real project, not answering from generic FastAPI patterns.
Kanban board	frontend/index.html with ToDo, InProgress, and Done columns; priority sorting; fetch/render logic; loading, empty, ready, and error states; drag-and-drop status updates.	Turns the Module 2 backend into a visible product surface students can interact with in the browser.
Create/edit modal	New Task and Edit flows with title trimming, POST/PATCH logic, server 422 messages, and dismissal behavior.	Connects frontend form behavior to backend validation and business rules.
Behavior contract + refactor	An 8-item behavior checklist, a pre-refactor commit, selected refactors, and the contract still passing afterward.	Prevents AI cleanup from silently changing behavior.
Expanded backend tests	Focused edge-case pytest tests for PATCH behavior, with at least one demonstrated through deliberate source breakage.	Shows that generated tests actually catch the bugs they claim to catch.
Debugging log and reflection	What failed, what evidence was used, what the AI suggested, and why the fix was accepted or rejected.	Builds the habit of evidence-based AI-assisted debugging.

Part 1 - From Backend API to Product UI

Key takeaway

Module 3 makes the API visible. Correctness now includes browser behavior, DevTools evidence, frontend state, and the backend rules from Module 2.

Module 2 gave you the backend foundation: a Task Tracker API, task fields, CRUD routes, status-transition rules, and tests. Module 3 uses that same repository and turns it into something a user can see and manipulate.

The important shift is not just visual. A frontend adds new ways to be wrong. A card can appear in the wrong column, a failed PATCH can leave the UI in a misleading state, a form can close after a server error, or an AI refactor can make code cleaner while breaking drag-and-drop. The module teaches you to catch those problems with the same loop used throughout the course: ask -> inspect -> run -> test -> refine.

What changes from Module 2 to Module 3

Module 2	Module 3
Backend API: data model, CRUD routes, status rules, and pytest coverage.	Frontend product surface: Kanban board, cards, drag-and-drop, modal form, and browser behavior.
Verification used curl, Swagger, and pytest.	Verification uses the browser, DevTools, network requests, console output, and pytest.
The key question was: does the endpoint return the right response?	The key question is: does the UI show the right state while respecting the API?
AI helped create backend code in controlled steps.	AI helps inside the coding flow: inline suggestions, chat, selected edits, and diff review.

Source-of-truth rule

The backend still owns validation and business rules. A nice-looking board is not correct if it allows invalid status values, ignores 422 responses, or hides backend errors.

Part 2 - Set Up VS Code + GitHub Copilot

Key takeaway

Setup is not complete when the assistant answers. Setup is complete only when you verify the answer against the real files in the repo.

Part 3.1 sets up VS Code and the GitHub Copilot workflow used in the rest of the module. The hands-on check is intentionally small: open app/main.py, ask the assistant to summarize the routes, and compare the answer with the decorators and functions in the file.

Setup checklist

Step	What to do	What to verify
1. Install tools	Use VS Code with the GitHub Copilot extension or the course-supported VS Code AI assistant.	The assistant is available for inline suggestions and chat.
2. Sign in	Authenticate with GitHub if prompted.	The extension is active and not stuck in a sign-in state.
3. Open the right folder	Open the Module 2 Task Tracker repo itself, not a parent folder that contains several projects.	The file explorer shows app/main.py, tests, and the project files directly.
4. Try three modes	Use inline suggestions, chat, and inline chat/edit on a selected section.	You know when the assistant is completing text, answering a question, or proposing an edit.

Step	What to do	What to verify
5. Verify context	Ask for a route summary from app/main.py and check every claim manually.	Each listed route has a matching decorator and function. Anything not present should be identified as not present.

First verification prompt

```
Read the selected app/main.py file.
Summarize only the API routes actually defined in this file.
For each route, include the method, path, and function name.
Also say whether CORS middleware is present.
If something is not present, say "not present" instead of guessing.
```

How to inspect the answer

If Copilot says...	Your check
A route exists	Find the matching @app.get, @app.post, @app.patch, or @app.delete decorator in app/main.py.
CORS is configured	Look for CORSMiddleware or app.add_middleware. If you cannot find it, the assistant guessed.
The repo has extra features such as authentication or users	Search the actual files. If those features are not present, correct the assistant and re-anchor it to the selected file.
The answer sounds generic	Check that VS Code is opened on the right folder and that the relevant file is selected or visible.

This part teaches the main rule for the rest of the module: the assistant can be fast and useful, but you verify before trusting.

Part 3 - Build the Kanban Board

Key takeaway

Build the board in layers. Static layout first, then styling, then fetch/render logic, then UI states, then drag-and-drop.

Part 3.2 builds frontend/index.html on top of the existing backend. The module uses a simple vanilla JavaScript frontend so the AI-assisted workflow stays visible: students can see the HTML, CSS, fetch calls, render logic, and event handlers in one place.

Board requirements

Requirement	What it means
Three status columns	Render columns for the exact API statuses: ToDo, InProgress, and Done. The UI may display them as To Do, In Progress, and Done.
Priority sorting	Within each column, sort cards High -> Medium -> Low. If priorities tie, keep a predictable secondary order such as lower id first.
Backend data	Load tasks with GET /tasks from the FastAPI backend, typically running at localhost:8000 during the demo.
UI states	Show loading while fetching, empty placeholders for columns without cards, the normal ready board when data exists, and an error state when the fetch fails.
Drag-and-drop	After the static board works, use browser-native drag-and-drop to move cards between columns. No external drag library is required for the module.
Server persistence	On drop, send PATCH /tasks/{id} with the new status. The backend decides whether the move is valid.
Rejected moves	If the server returns 422 or the network fails, refresh or revert the UI and show the user an error message.

Recommended build sequence

1. Create frontend/index.html and open it with a local static server such as Live Server.
2. Write the static board skeleton: page header, three columns, and one sample card.
3. Use inline suggestions for repeated HTML and CSS patterns, but inspect each suggestion before keeping it.
4. Ask chat for a small fetch/render plan before asking for implementation.
5. Add fetchTasks() and renderBoard() so real tasks from GET /tasks appear in the correct columns.
6. Add loading, empty, ready, and error states before adding drag-and-drop complexity.
7. Use DevTools to check failed requests, status codes, and response bodies.
8. Add drag-and-drop only after the board renders correctly from real backend data.

Useful constants from the demo

```
API_BASE = "http://localhost:8000"
STATUSES = ["ToDo", "InProgress", "Done"]
STATUS_LABELS = { ToDo: "To Do", InProgress: "In Progress", Done: "Done" }
PRIORITY_ORDER = { High: 0, Medium: 1, Low: 2 }
```

CORS note from the recording

The frontend and backend run on different local origins during the demo. For example, the frontend might be served from localhost:5500 while the FastAPI backend runs on localhost:8000. If the browser console says a request was blocked by CORS policy, update the FastAPI app to allow the local frontend origin. The recording includes this as a practical troubleshooting step, not as optional trivia.

Typical local origins to consider include http://localhost:5500, http://127.0.0.1:5500, and any frontend dev-server origin your environment uses. Add only what your local setup needs and keep the change visible in app/main.py.

Board verification checklist

Behavior to verify	Evidence to use
Three columns render	Browser view shows To Do, In Progress, and Done columns with correct counts or clear empty states.
Priority sort works	Create or seed High, Medium, and Low tasks and confirm the visual order inside each column.
Loading state appears	Reload while the request is pending or use DevTools throttling to see the loading behavior.
Empty state appears	A column with no tasks still has a clear placeholder and remains visually understandable.
Error state appears	Stop the backend and reload. The UI should show a clear error instead of a blank board.
Valid drag persists	Drag a card to a valid next status. DevTools shows PATCH /tasks/{id} and the server returns success.
Rejected drag is handled	Try an invalid transition such as moving a Done task back to ToDo if the backend rejects it. The card should revert or refresh, and the UI should show the server message.
Same-column drag is not noisy	Dropping a card back into its original column should not send unnecessary PATCH requests.

Part 4 - Build the Create / Edit Modal Form

Key takeaway

The modal is where frontend convenience meets backend rules. It should make creation and editing easy without bypassing validation.

Part 3.3 extends the board with a New Task button and Edit buttons on task cards. The instructor emphasizes focused selected edits: choose the form or submit handler and ask the assistant to change only that section instead of rewriting the whole file.

Modal elements

Element	Purpose
New Task button	Opens the modal in create mode with empty/default values.
Edit button on each card	Opens the modal in edit mode and fills the form from the selected task.
Fields	Title, description, status, priority, assignee, and a hidden or internal task id for edit mode.
Save action	Uses POST /tasks for create mode and PATCH /tasks/{id} for edit mode.
Cancel, close button, Escape, and overlay click	Dismiss the modal and clear stale form state.
Error display	Shows client validation errors and server 422 messages without pretending the save succeeded.

Create mode vs. edit mode

Create mode	Edit mode
No existing task id. The form starts with defaults such as ToDo and Medium unless the UI chooses different defaults.	Existing task id is known. The form is prefilled with the current task values.
Submit sends POST /tasks with the new task payload.	Submit sends PATCH /tasks/{id} with updated fields.
On success, the board refreshes and the new card appears in the correct column and priority slot.	On success, the board refreshes and the edited card moves or reorders if status or priority changed.
If title is empty after trim, no network request should be sent.	If the backend rejects an invalid transition, the modal stays open and shows the server message.

Validation rules to preserve

- Client-side title validation: trim whitespace before checking whether the title is empty.
- Do not send a request when the title is empty or whitespace-only.
- Empty assignee can be sent as null if the backend expects optional assignee data.
- Server-side 422 responses should keep the modal open and show a useful message.
- A successful create or edit should clear errors, close the modal, and refresh the board.

Five flows to verify

Flow	Expected behavior
1. Empty title	Submit with an empty or whitespace-only title. No request fires, and the title error is visible.
2. Create task	Create a High-priority ToDo task. POST succeeds, and the card appears in the ToDo column above lower-priority cards.
3. Edit task	Change priority or status. PATCH succeeds, and the card reorders or moves to the correct column.
4. Invalid transition	Try a backend-rejected status change. PATCH returns 422, the modal stays open, and the server message is visible.
5. Dismissal	Cancel, X/close, Escape, and overlay click all close the modal and clear stale values or errors.

Debug form failures from evidence

When the modal fails, do not prompt the assistant with "it does not work." Use the action you took, the network request, the status code, the response body, and what the UI showed. Evidence makes the AI more useful and keeps you in control.

Part 5 - Refactor What You Just Built

Key takeaway

Refactoring with AI is risky because clean-looking code can quietly change behavior. Write the contract first, then refactor.

Part 3.4 teaches safe AI-assisted refactoring. The instructor does not treat a refactor as a cosmetic step. The goal is to improve readability, structure, or visual polish while proving that the board still behaves the same.

Safe refactor workflow

1. Run the app and mark each contract item PASS before refactoring. If a behavior already fails, fix it first.
2. Commit or otherwise checkpoint the working state so rollback is easy.
3. Select one section at a time: render logic, drag handlers, modal functions, submit handler, or CSS block.
4. Ask the assistant for a focused cleanup of the selected section only.
5. Inspect the diff before accepting. A diff is a proposal, not a decision.

6. Run the browser and re-check the contract after every accepted change.
7. If something breaks, select the smallest broken section and fix that specific regression. Do not ask for a full-file rewrite.

Diff review red flags

Red flag	Why it matters
Status strings changed	Sending "In Progress" instead of "InProgress" can break the API contract.
URLs or methods changed	GET /tasks and PATCH /tasks/{id} must stay aligned with the backend.
Class names or data attributes removed	Rendering, CSS, or event handling may depend on them.
.trim() removed from title validation	Whitespace-only titles can slip through the client layer.
422 handling or revert path removed	The UI may appear to save a change the server rejected.
Event listeners moved carelessly	Re-rendering with innerHTML can remove listeners if the code does not reattach or delegate correctly.
The assistant rewrites the whole file	Large diffs are harder to inspect and more likely to introduce hidden regressions.

The transferable habit is simple: contract first, checkpoint second, selected refactor third, verification after every accepted change.

Part 6 - Test and Debug What You Just Built

Key takeaway

AI can help write tests, but tests earn trust only when you prove they fail for the bug they claim to catch.

Part 3.5 returns to the backend because the frontend depends on backend validation. The demo uses Copilot Chat to brainstorm missing PATCH edge cases, then adds one focused test at a time. The instructor demonstrates deliberate breakage: temporarily change the source so the new test should fail, confirm it fails, restore the correct source, and confirm it passes again.

Candidate PATCH edge cases from the lecture

Scenario	Expected result
Unsupported status such as "Archived"	422 validation error.
Invalid priority such as "Urgent"	422 validation error.
Invalid transition from InProgress back to ToDo	422 invalid status transition.
Valid transition from InProgress to Done	200 success and updated status.
PATCH to a task id that does not exist	404 not found.

Scenario	Expected result
Empty JSON body or unsupported update shape	Backend-defined validation response; confirm against the actual current model and tests.

The test verification loop

1. Ask the assistant to brainstorm edge cases first. Do not request code yet.
2. Pick one scenario and generate one pytest test in tests/test_tasks.py using the existing fixture pattern.
3. Run the targeted test with pytest -k <test_name> -v. It should pass on correct source code.
4. Deliberately break the source code that the test is supposed to protect.
5. Run the test again. It must fail for the expected reason. If it still passes, the assertion is weak or wrong.
6. Restore the source code and rerun the test. It should pass again.
7. Repeat for each new edge-case test and then run the full suite.

Lecture example: invalid backward transition

The visible testing example creates a task, moves it from ToDo to InProgress, then tries to move it back to ToDo. The correct backend behavior is a 422 response with an invalid-transition message. To prove the test is meaningful, the demo temporarily allows that transition in the source, watches the test fail because it receives 200 instead of 422, then restores the business rule.

```
# Behavior being tested, not exact required code
create task -> status is ToDo
PATCH status to InProgress -> expect 200
PATCH status back to ToDo -> expect 422
assert the response body mentions an invalid status transition
```

Debug from failure output

For debugging, introduce or encounter a subtle bug, run pytest, and paste the full failure output into chat. Ask for the root cause, the file and line likely involved, why the failure happens, and the source-code fix. Do not ask the assistant to change the test unless the test itself is demonstrably wrong.

Cause fix vs. symptom suppression

Reject symptom suppression	Accept cause fixes
Changing an assertion from 422 to 200 just because the current code returned 200.	Restoring the transition rule so invalid backward movement returns 422 again.
Deleting the assertion on the response body.	Keeping the assertion and fixing the source so the expected message is returned.

Reject symptom suppression	Accept cause fixes
Adding try/except around a failing call to hide the error.	Fixing the missing await, wrong field name, wrong status code, or incorrect condition that caused the failure.
Changing tests broadly without explaining the root cause.	Making the smallest source change and explaining why it addresses the failure.

Debugging log format

Four-line log

Line	What to write
1. Bug or failure	What you broke or what failed, stated briefly.
2. Evidence	Failing test name plus the key status code, response body, or traceback line.
3. AI diagnosis	The assistant's stated root cause and proposed source fix.
4. Decision	Accept or reject, with one sentence explaining why it is a cause fix or symptom suppression.

Part 7 - Common Pitfalls and Recovery Habits

Key takeaway

Most Module 3 mistakes come from trusting output before checking context, browser evidence, or the behavior contract.

Pitfall guide

Pitfall	How to recover
Wrong repo or file context	Open the Task Tracker repo folder directly. Select the relevant file and ask the assistant to reread only that file.
Building too much at once	Back up to one layer: static board, styling, fetch/render, UI states, drag-and-drop, then modal.
Full-file AI rewrites	Reject or narrow the request. Select a function or section and preserve URLs, statuses, selectors, and behavior.
Refactor regression	Use the 8-item contract to identify the exact broken behavior, then fix only that section.
Generated test that is too weak	Break the source. If the test still passes, improve the assertion before trusting it.

Pitfall	How to recover
Symptom-suppression debugging	Reject fixes that weaken tests or hide errors. Ask for a source-cause explanation and a minimal source fix.

Part 8 - What You Should Have at the End of Module 3

Key takeaway

The final deliverable is not just a working UI. It is a working UI plus evidence that you inspected, tested, refactored, and debugged responsibly.

End-of-module checklist

You should have...	Check
VS Code + Copilot working on the right repo	You verified at least one answer against app/main.py.
A Kanban board in frontend/index.html	Three columns, priority sorting, fetch/render logic, and all four UI states.
Drag-and-drop status updates	Valid moves persist through PATCH; invalid moves revert or refresh and show an error.
A create/edit modal	Title trimming, POST/PATCH behavior, server 422 handling, and dismissal flows all verified.
A behavior contract	The 8-item checklist passed before and after the refactor.
A cleaner/refactored frontend	The refactor was selected, inspected, accepted deliberately, and verified.
Expanded backend tests	New edge-case tests were run and at least one was proven through deliberate breakage.
A debugging log and reflection	You recorded evidence, AI diagnosis, decision, and a brief comparison of AI tools/workflows used so far.